# How to Add a New Agent/Tool to KnowThyself

This guide shows you—step by step—how to add a new agent (aka “tool agent”) that the Orchestrator LLM can call. It matches the existing patterns used by `transformerlens_agent`, `bias_evaluation_agent`, and `bertviz_agent`.

---

## 0) Mental model

- **Supervisor / Orchestrator LLM** routes user requests to agents.

**Agents** are plain Python callables with the signature:

```
def some_agent(state: AgentState, model_manager: ModelManager) -> dict:
    return {"messages": [AIMessage(...)]}
```

- 
- **ModelManager** gives you a ready-to-use model backend (`hooked`, `huggingface`, `ollama`, etc).
- **AgentState** lets you read the latest human message (via `get_most_recent_human_message`).
- Agents **must** return a list of `AIMessage`s; any structured payload for the UI (e.g., paths, arrays) goes in `additional_kwargs`.

---

## 1) Where files live

Create your tool under:

```
src/knowthyself/tools/<your_tool_name>/
    __init__.py
    <your_tool>.py          # your agent implementation
    prompts.py              # (optional) prompt templates
    utils.py                # (optional) helper functions
    requirements.txt        # (optional) extra deps
```

If your agent writes artifacts (images/HTML/CSVs), save them under:

src/files/documents/results/

and return the final path via `additional_kwargs`.

---

## 2) Agent function contract

**Input**

- `state: AgentState` — use `get_most_recent_human_message(state)` to get the user text.
- `model_manager: ModelManager` — call `set_user_model(backend=..., model_name=...)` and `get_user_model()`.

**Output**

- `{"messages": [AIMessage(...)]}`
    - `AIMessage.content` is human-readable text.
    - Put machine-readable outputs in `additional_kwargs` (e.g., `{"plot_path": "...", "scores": {...}}`).
    - Use `type="ai"`.

**Do**

- Parse user intent/arguments with a `PydanticOutputParser` + `PromptTemplate`.
- Fail **gracefully**: catch exceptions and return a single `AIMessage` that explains the failure briefly.
- Keep prompts short and deterministic; include strict `format_instructions`.

---

## 3) Choosing a model backend

Pick the backend that fits your tool:

| Backend | `set_user_model` example | Use cases |
|---|---|---|
| Hooked (TransformerLens) | `set_user_model(backend="hooked", model_name="gpt2-small")` | attention, circuits, mech-int |
| HuggingFace | `set_user_model(backend="huggingface", model_name="distilbert-base-uncased")` | embeddings, tokenization, HF heads/attn |
| Ollama | `set_user_model(backend="ollama", model_name=os.getenv("OLLAMA_MODEL","llama3:8b-instruct"))` | local LLM inference, scoring, eval |

Then fetch with:

USER_MODEL = model_manager.get_user_model()

(Your project currently returns a dict like `{"model": ..., "tokenizer": ...}` for HF; other backends may return a wrapper object—follow existing agents for patterns.)

---

# 4) Prompt + schema pattern (recommended)

- Define a minimal prompt that **forces JSON** via `parser.get_format_instructions()`.
- Use a small `BaseModel` to validate/normalize inputs.
- Optionally backfill numbers with regex from raw user text.

**Example:**

```
class MyArgs(BaseModel):
    target: str
    threshold: float | None = None

    @validator("target", pre=True)
    def clean_target(cls, v):
        return str(v or "").strip()
```

```
EXTRACTION_PROMPT = (
    "Return ONLY the JSON described by {format_instructions} with fields:\n"
    "- target: string\n- threshold: number or null\n\n"
    "User request:\n{messages}"
)
```

---

# 5) Minimal agent template (copy/paste)

```
# src/knowthyself/tools/mytool/mytool.py
import os, re, json
from typing import Any, Dict, Optional
from pydantic import BaseModel, validator
from langchain.output_parsers import PydanticOutputParser
from langchain.prompts import PromptTemplate
from langchain_core.messages import AIMessage

from src.knowthyself.utils.graph_utils import get_most_recent_human_message, AgentState
from src.knowthyself.utils.model_manager import ModelManager

# ---------- 1) Argument extraction ----------
EXTRACTION_PROMPT = (
    "You are an extractor. Return ONLY the JSON described by {format_instructions} with
fields:\n"
    "- target: string\n"
    "- threshold: number or null\n\n"
    "If no threshold is given, use null.\n\n"
    "User request:\n{messages}"
)

class ExtractArgs(BaseModel):
    target: str
    threshold: Optional[float] = None

    @validator("target", pre=True)
    def _clean(cls, v: Any):
        s = str(v or "").strip()
        return s or "default"

def _extract_args(user_message: str, llm) -> Dict[str, Any]:
    parser = PydanticOutputParser(pydantic_object=ExtractArgs)
    prompt = PromptTemplate(
        template=EXTRACTION_PROMPT,
        input_variables=["messages"],
        partial_variables={"format_instructions": parser.get_format_instructions()},
    )
    raw = (prompt | llm).invoke({"messages": user_message})
```

```
    text = getattr(raw, "content", raw)
    try:
        obj: ExtractArgs = parser.parse(text)
        return obj.dict()
    except Exception:
        return {"target": user_message.strip() or "default", "threshold": None}

# ---------- 2) The agent ----------
def mytool_agent(state: AgentState, model_manager: ModelManager) -> dict:
    """
    Demo agent:
    - extracts (target, threshold)
    - loads a backend model
    - computes some result
    - returns an AIMessage with explanation + machine-readable payload
    """
    MAIN_LLM = model_manager.get_orchestrator()

    # Choose a backend and model
    model_name = os.getenv("HUGGINGFACE_MODEL", "distilbert-base-uncased")
    try:
        model_manager.set_user_model(backend="huggingface", model_name=model_name)
        user_model = model_manager.get_user_model()  # e.g., {"model": hf_model,
"tokenizer": hf_tok}
    except Exception as e:
        return {"messages": [AIMessage(content=f"Failed to load model '{model_name}': {e}",
type="ai")]}

    user_text = get_most_recent_human_message(state) or ""
    args = _extract_args(user_text, MAIN_LLM)
    target, threshold = args["target"], args["threshold"]

    # ---- Your core logic here (replace this) ----
    # For example: compute an embedding or a classification score.
    try:
        # placeholder result
        result = {"target": target, "score": 0.73, "threshold": threshold}
    except Exception as e:
        return {"messages": [AIMessage(content=f"Computation failed: {e}", type="ai")]}

    # Ask orchestrator to explain the numbers (optional but user-friendly)
    try:
        explanation = MAIN_LLM.invoke(
            "Explain this JSON result briefly for a user (2–3 sentences):\n" + json.dumps(result,
indent=2)
        )
        explanation_text = getattr(explanation, "content", explanation)
    except Exception:
```

```
        explanation_text = "Computed a result. See details in the attached metadata."

    return {
        "messages": [
            AIMessage(
                content=explanation_text,
                additional_kwargs={"tool": "mytool", "result": result, "model_name": model_name},
                type="ai",
            )
        ]
    }
```

---

# 6) Registering / exposing the agent

Depending on your router setup, you typically:

1. **Export** the agent in the module's `__init__.py`:

```
# src/knowthyself/tools/mytool/__init__.py
from .mytool import mytool_agent
__all__ = ["mytool_agent"]
```

2. **Wire it into the router/graph** where other tools are mapped (e.g., a dict like `{"transformerlens": transformerlens_agent, "bertviz": bertviz_agent, ...}`).
   For example:

```
# src/knowthyself/agents/registry.py (example)
from src.knowthyself.tools.mytool import mytool_agent
AGENT_REGISTRY["mytool"] = mytool_agent
```

3. **Add routing logic** in your supervisor policy (e.g., keyword match, classifier, or explicit command) so the Orchestrator picks `mytool`.

---

# 7) Returning artifacts (images/HTML/CSVs)

- Save to: `src/files/documents/results/<meaningful_name>.<ext>`

Return the absolute path in `additional_kwargs`, e.g.:

additional_kwargs={"plot_path": "/.../src/files/documents/results/plot.png"}

- 
- Keep `AIMessage.content` short and human-friendly; **do not** dump huge data in `content`.

---

## 8) Error handling & logging

- Wrap all I/O/model calls in `try/except` and return one `AIMessage` explaining the failure succinctly.
- Prefer actionable errors (“Check that model X is installed and Ollama is running at :11434”).
- Use `print()` or your preferred logger for diagnostic lines; never crash the agent.

---

## 9) Testing checklist

- **Unit**: Extractor schema parses representative prompts; edge cases return safe defaults.
- **Backend**: `set_user_model` + `get_user_model` returns what your agent expects.
- **Run**: Trigger agent with sample user text; confirm one `AIMessage` returns.
- **Artifacts**: Files are saved and readable; UI keys present in `additional_kwargs`.
- **Bad paths**: Kill model server / remove weights—ensure graceful failure.
- **Latency**: Keep heavy work optional or batched; stream only if your infra supports it.

---

## 10) Common pitfalls

- Not forcing strictly formatted JSON → parser fails. Always include `format_instructions`.
- Writing artifacts to non-existent dirs → `os.makedirs(..., exist_ok=True)`.
- Assuming a single backend shape. Different backends return different objects—mirror working agents.
- Overloading `content` with raw JSON blobs. Put them in `additional_kwargs` instead.

---

## 11) Quick start: duplicate an existing agent

- For **attention/visualization** → copy `bertviz_agent`.
- For **scoring/evaluation** → copy `bias_evaluation_agent`.
- For **mech-interp** → copy `transformerlens_agent`.

Rename, swap the extractor schema + core logic, and register it.

---

## 12) Style guide (brief)

- Keep functions small and single-purpose.
- Put prompts and schemas near the agent (or in `prompts.py` if shared).
- Prefer explicit names: `mytool_agent`, `extract_mytool_args`, `MYTOOL_PROMPT`.
- Return exactly one `AIMessage` per agent call (unless you truly need multiple).

---

### That's it!

Use the template, wire it into the registry, and your new tool will behave like the built-ins—parse → run → explain → return a clean `AIMessage` with any artifacts attached.